Spring Data JPA: Core & Repositories

1. What is Spring Data JPA?

A: Spring Data JPA is a framework that makes it easy to use the **Java Persistence API (JPA)** in Spring applications. It eliminates much of the boilerplate code required for data access by automatically creating repository implementations based on simple method names defined in interfaces.

2. Explain the main features of Spring Data JPA.

A: Main features include **Automatic Repository Generation** (no implementation needed for basic CRUD methods), **Query Derivation** (writing queries by method name), and integrated support for **Pagination** and **Sorting**. It also simplifies the use of custom queries via the @Query annotation and provides built-in transaction management.

3. How do you create a custom repository interface in Spring Data JPA?

A: You create a custom repository by defining an interface that extends either **CrudRepository** or **JpaRepository**. The interface uses generics to specify the Entity type and the type of the Entity's primary key. You then declare method names that follow specific conventions (e.g., findByLastName(String lastName)).

4. What is the difference between CrudRepository and JpaRepository?

A: CrudRepository provides only basic CRUD functionality (Create, Read, Update, Delete). **JpaRepository** extends CrudRepository and adds more advanced capabilities specific to JPA, such as bulk operations, pagination, sorting, and flushing changes to the database. JpaRepository is the preferred choice for complex applications.

5. Explain a few commonly used CrudRepository methods.

A: save() is used to persist a new entity or update an existing one. **findById(ID id)** fetches an entity by its primary key. **findAll()** retrieves all entities of that type. **deleteById(ID id)** removes a single entity based on its primary key.

6. What are the rules to follow when declaring custom methods in a Spring Data repository?

A: The methods must start with keywords like **find...By**, **count...By**, **delete...By**, or **exists...By**. The part after the keyword must reference valid, existing entity properties (e.g., findByEmailAddressAndActive(String email, boolean active)). The method signature must match the entity properties and required return type.

7. What is the purpose of the save() method in CrudRepository?

A: The **save()** method serves a dual purpose: it is used to **persist a new entity** to the database (if the primary key is null or unset), and it is used to **update an existing entity** (if the primary key already has a value that corresponds to a record in the database).

8. How do you write a custom query in Spring Data JPA?

A: You write a custom query by placing the **@Query** annotation above the method declaration in your repository interface. Inside the annotation, you can define a **JPQL (Java Persistence Query Language)** query or a native SQL query by setting the nativeQuery = true attribute.

9. What is the use of the @Modifying annotation in Spring Data JPA?

A: The **@Modifying** annotation is required for any custom query that changes the data in the database (i.e., INSERT, UPDATE, or DELETE queries). It tells Spring Data that the method is not a read operation and should manage transaction synchronization and session clearing appropriately.

10. What is the use of the @Id annotation?

A: The **@Id** annotation is used to mark a specific field in an entity class as the **primary key** of that entity. This key is used by JPA to uniquely identify the entity instance in the database and within the persistence context.

11. How will you create a composite primary key in Spring Data JPA?

A: You create a composite primary key by defining a separate class for the key (the **ID class**) that implements Serializable and contains the key fields. The entity then uses the **@IdClass** annotation to reference this ID class, and the key fields within the entity are marked with **@Id**.

12. What is the use of the @Temporal annotation?

A: The **@Temporal** annotation is used on fields of type java.util.Date or java.util.Calendar. It specifies the exact type of the corresponding database column: TemporalType.DATE, TemporalType.TIME, or TemporalType.TIMESTAMP. This ensures the conversion between Java and SQL types is accurate.

13. What is the difference between FetchType.EAGER and FetchType.LAZY?

A: FetchType.EAGER means the associated entity data is loaded from the database immediately when the parent entity is loaded. **FetchType.LAZY** means the associated entity data is only loaded from the database when it is explicitly accessed by the application code for the first time. Lazy fetching improves performance by delaying unnecessary I/O.

14. What is pagination, and how do you implement pagination in Spring Data?

A: Pagination is the process of retrieving data in small, sequential chunks (pages) instead of retrieving all data at once. You implement it by passing a **Pageable** object to the repository method (e.g., findAll(Pageable pageable)). This returns a **Page** object, which contains the list of entities plus metadata like total pages and total elements.

15. Write a query method for sorting in Spring Data JPA.

A: You implement sorting by appending the OrderBy keyword to the method name, followed by the property name and the direction.

Example Query Method:

List<User> findByStatusOrderByLastNameAsc(String status);

Advanced JPA Concepts & Performance

16. Explain the @Transactional annotation in Spring.

A: The **@Transactional** annotation tells Spring to wrap a method's execution within a database transaction. Spring ensures that if the method completes successfully, the transaction is committed; if an exception occurs, the transaction is automatically rolled back. This guarantees **Atomicity** for all database operations within the method.

17. What is the difference between findById() and getOne() in Spring Data JPA?

A: findById() (or getReferenceById() in newer versions) executes a query immediately and returns the real entity or Optional.empty() if not found. **getOne()** (deprecated) used to return a proxy of the entity immediately without hitting the database, only fetching the data when a getter method was called. findById() is the standard method for retrieval.

18. What is the difference between delete() and deleteInBatch() methods?

A: delete() iterates through entities one by one, executing a separate DELETE statement for each entity and firing lifecycle events for every one. **deleteInBatch()** executes a single, bulk DELETE SQL statement, which is much faster but skips the lifecycle events and does not interact with the persistence context (first-level cache).

19. What is the use of the @EnableJpaRepositories annotation?

A: The **@EnableJpaRepositories** annotation is used on a configuration class to tell Spring Data where to find and scan the repository interfaces. It is essential in non-Spring Boot applications or when the repositories are located outside the default package structure.

20. Explain Query By Example (QBE) in Spring Data JPA.

A: Query By Example (QBE) is a user-friendly API for building simple, dynamic queries based on an example entity instance. You create an instance of your entity (the "probe") and set the fields you want to match. Spring Data automatically generates the query using the field values provided, supporting easy matching and ignoring null fields.

21. What are projections in Spring Data JPA, and when would you use interface-based vs class-based projections?

A: Projections allow you to retrieve only a subset of fields from an entity, avoiding the overhead of loading the full object. Use **interface-based projections** (closed or open) when you need a simple contract with only getter methods. Use **class-based projections** when you need to perform calculations or custom logic inside a DTO constructor.

22. How does Spring Data JPA handle named queries, and how do you define and use them?

A: Spring Data JPA handles named queries by searching for methods annotated with **@NamedQuery** or by looking for external definition files (like orm.xml). You define them using the **@NamedQuery** annotation on the entity class. You use them by creating a repository method with the exact same name as the defined named query.

23. What is the difference between using @Query with JPQL vs using a native query? When would you choose each?

A: JPQL (Java Persistence Query Language) uses entity names and fields (object-oriented); it is database-agnostic. A **native query** uses raw, database-specific SQL syntax. Choose **JPQL** for standard queries to maintain portability. Choose a **native query** when you need to use database-specific features (like special functions or complex hints) that JPQL does not support.

24. What is Specification in Spring Data JPA, and how does it help in building dynamic queries?

A: Specification is a type-safe object that represents a single query predicate (a condition, like "status = active"). It helps in building **dynamic queries** by allowing the application logic to build a list of predicates at runtime, which are then combined to form a complex WHERE clause.

25. How can you combine multiple Specification instances to build complex filters?

A: You combine multiple Specification instances using logical operators provided by the Specification interface, such as **and()** and **or()**. This allows the application to build up complex WHERE clause logic (e.g., specification.where(isAvailable()).and(hasHighPrice())) by chaining the reusable predicates.

26. What is the difference between persist(), merge(), and remove() in JPA, and how does Spring Data abstract these operations?

A: persist() adds a new entity to the context (INSERT). **merge()** copies the state of a detached entity onto a managed entity (UPDATE or INSERT). **remove()** schedules a managed entity for

deletion (DELETE). Spring Data abstracts these via the **save()** (for persist/merge) and **delete()** methods.

27. What is the N+1 select problem in JPA, and how can you reduce it using Spring Data JPA?

A: The **N+1 select problem** occurs when loading a parent entity (1 query) and then executing a separate query for each of its N associated child entities, leading to N+1 total queries. You reduce it by using **fetch joins** or **@EntityGraph** to force JPA to retrieve all required child data in a single, optimized query.

28. How can you use fetch joins or @EntityGraph to optimize queries in Spring Data JPA?

A: You use **fetch joins** by adding the JOIN FETCH clause to a custom JPQL query defined with @Query. You use **@EntityGraph** by placing the annotation above the repository method and listing the exact association paths (relationships) that JPA should eagerly load in the initial query. Both force eager loading to reduce N+1 problems.

29. How does the first-level cache (persistence context) impact queries and updates in JPA?

A: The **first-level cache** (Persistence Context) acts as a temporary memory area for entities loaded during the current transaction. When you query for an entity, JPA first checks this cache. Updates impact it because any changes to a managed entity in this context are automatically tracked and flushed to the database when the transaction commits.

30. How do you enable auditing in Spring Data JPA (e.g., @CreatedDate, @LastModifiedDate)?

A: You enable auditing by adding two main elements: 1) Annotate your base entity class fields with **@CreatedDate** and **@LastModifiedDate**. 2) Add the **@EnableJpaAuditing** annotation to a configuration class in your application.

31. What is the purpose of @EnableJpaAuditing, and what additional setup is required?

A: The purpose of **@EnableJpaAuditing** is to activate the auditing feature within Spring Data JPA, enabling automatic management of fields like creation date and modification date. The additional setup required is providing a custom bean that implements **AuditorAware<T>** if you need to automatically track the ID or name of the user who made the change.

32. How would you implement soft delete in a Spring Data JPA application?

A: I would implement soft delete by adding a boolean column (e.g., is_deleted) to the entity. Instead of using the standard delete() method, I would create a custom method (e.g., softDeleteById()) that updates the is_deleted flag to true. I would then use **@Where(clause = "is_deleted = false")** to filter out deleted records globally.

33. How can you globally filter out soft-deleted records using JPA or Hibernate features?

A: You can globally filter out soft-deleted records by using the **@Where(clause = "is_deleted = false")** annotation at the entity class level (a Hibernate feature). Alternatively, you can use a generic **JPA Filter** (defined in XML or Java config) that applies a SQL condition to all queries against that entity, automatically excluding soft-deleted rows.

34. How does Spring Data JPA behave inside a @Transactional boundary for read and write operations?

A: Inside a @Transactional boundary, Spring Data JPA operations work within the same **Persistence Context** (first-level cache). For **read** operations, the context ensures data consistency within the transaction. For **write** operations (save(), delete()), changes are tracked and only executed as a single batch when the transaction successfully **commits**.

35. In which scenarios would you change the isolation level or propagation of a transactional method that uses Spring Data JPA?

A: You would change the **Isolation Level** for critical concurrent operations (like inventory management) that require stricter rules than the default (e.g., changing to REPEATABLE_READ to prevent data modification). You change **Propagation** when a method must run in its own

independent transaction regardless of whether a calling transaction exists (e.g., using REQUIRES_NEW for logging).

36. How do you configure Spring Data JPA to connect to multiple databases in the same application?

A: You configure multiple databases by defining separate configuration classes for each database. Each class must manually define and configure three unique beans: the **DataSource** (connection details), the **LocalContainerEntityManagerFactoryBean** (maps entities to that DB), and the **JpaTransactionManager** (manages transactions).

37. How do you map different repositories to use different EntityManagerFactory beans?

A: You map different repositories by using the **@EnableJpaRepositories** annotation on the configuration class for each database. Within this annotation, you specify the **entityManagerFactoryRef** and **transactionManagerRef** attributes, pointing them to the specific, uniquely named bean for that particular database connection.